Exploratory Testing for Charter II

Session Participants

Nathan Audegond 261171251 — nathan.audegond@mail.mcgill.ca (Session lead)

Kevin He 261167637 — kevin.he2@mail.mcgill.ca (Session participant)

Section 1: Session Summary

1.1: Session Charter

Identify capabilities and areas of potential instability of the "rest api todo list manager". Identify documented and undocumented "rest api todo list manager" capabilities. For each capability, create a script or small program to demonstrate the capability. Exercise each capability identified with data typical to the intended use of the application.

Session focus: Explore /projects, /projects/:id and /projects/:id/tasks|categories to discover capabilities and instabilities.

1.2: Version

Release version 1.5.5 of the Todo List Manager, running with profile version=3 (default data).

1.3: Set Up

Ubuntu 24.04 with Java 21. Requests were issued with curl from the session script so that every request can be replayed exactly; responses were captured in the session log. No Postman collection was used.

1.4: Build

Download from https://github.com/eviltester/thingifier/releases/tag/1.5.5 — runTodoManagerRestAPI-1.5.5.jar, started with: java -jar runTodoManagerRestAPI-1.5.5.jar (API and documentation at http://localhost:4567 and http://localhost:4567/docs).

Section 2: Session Notes

2.1: Session Date and Time

October 14, 2026. Duration: 45 minutes (timeboxed). Requests are numbered by step in Table 2 in the exact order they were run; the raw log preserves that order (curl logs do not carry per-request wall-clock times).

2.2: Exploration Notes

Table 1: Legend for request colours

Legend	
	As Expected
	Bug
	To Explore

Table 2: Session Notes taken during Exploratory Testing

Time	Endpoint Tested	Response	Reference File
Setup	**Run the application through the jar file** java -jar runTodoManagerRestAPI-1.5.5.jar	Application available at http://localhost:4567; default (profile version=3) data present.	runTodoManagerRestAPI-1.5.5.jar (course-provided; not included in this package)
Step 1	**Test API GET request** /projects Documented? DOCUMENTED Expected: 200 with the list of all projects	**Status 200: OK** Observed: HTTP 200 — {"projects":[{"id":"1","title":"Office Work","completed":"false","active":"false","description":"","tasks":[{"id":"2"}]}]} Result: **As Expected** Side note: Default project 1 present	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 2	**Test API POST request** /projects body: {"title":"thesis"} Documented? DOCUMENTED Expected: 201 created; defaults completed=false, active=false, description=""	**Status 201: Created** Observed: HTTP 201 — {"id":"202","title":"thesis","completed":"false","active":"false","description":""} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh

Time	Endpoint Tested	Response	Reference File
Step 3	**Test API POST request** /projects body: {"title":"launch","completed":true,"active":true,"description":"q4"} Documented? DOCUMENTED Expected: 201 created with all fields stored	**Status 201: Created** Observed: HTTP 201 — {"id":"203","title":"launch","completed":"true","active":"true","description":"q4"} Result: **As Expected** Side note: completed and active both true is accepted (consistency question)	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 4	**Test API POST request** /projects body: {"description":"no title"} Documented? DOCUMENTED Expected: By analogy with todos/categories, title should be mandatory: 400	**Status 201: Created** Observed: HTTP 201 — {"id":"204","title":"","completed":"false","active":"false","description":"no title"} Result: **Bug** Side note: Title is not mandatory for projects — inconsistent validation	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 5	**Test API POST request** /projects body: {"title":"x","completed":"yes"} Documented? DOCUMENTED Expected: 400: completed should be BOOLEAN	**Status 400: Bad Request** Observed: HTTP 400 — {"errorMessages":["Failed Validation: completed should be BOOLEAN"]} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 6	**Test API GET request** /projects/1 Documented? DOCUMENTED Expected: 200 with project 1	**Status 200: OK** Observed: HTTP 200 — {"projects":[{"id":"1","title":"Office Work","completed":"false","active":"false","description":"","tasks":[{"id":"2"}]}]} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 7	**Test API POST request** /projects/1 body: {"active":true} Documented? DOCUMENTED Expected: 200; POST amends only the given fields	**Status 200: OK** Observed: HTTP 200 — {"id":"1","title":"Office Work","completed":"false","active":"true","description":"","tasks":[{"id":"2"}]} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 8	**Test API PUT request** /projects/1 body: {"title":"Office Work"} Documented? DOCUMENTED Expected: 200; PUT replaces the project fields	**Status 200: OK** Observed: HTTP 200 — {"id":"1","title":"Office Work","completed":"false","active":"false","description":""} Result: **Bug** Side note: BUG-09: the PUT also drops the project's task relationships (see next row)	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 9	**Test API GET request** /projects/1/tasks Documented? DOCUMENTED Expected: After the PUT the previously linked tasks should still be listed	**Status 200: OK** Observed: HTTP 200 — {"todos":[]} Result: **Bug** Side note: BUG-09 evidence: empty list returned	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 10	**Test API POST request** /projects/1/tasks body: {"title":"todo created under project"} Documented? UNDOCUMENTED Expected: Creating a task directly under a project is not described in the guide	**Status 201: Created** Observed: HTTP 201 — {"id":"405","title":"todo created under project","doneStatus":"false","description":"","tasksof":[{"id":"1"}]} Result: **As Expected** Side note: Undocumented capability: works, 201, todo created and linked	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 11	**Test API POST request** /projects/1/tasks body: {"id":"1"} Documented? DOCUMENTED Expected: 201: link existing todo 1 to project 1	**Status 201: Created** Observed: HTTP 201 — (empty response body) Result: **As Expected** Side note: Empty response body	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 12	**Test API POST request** /projects/1/tasks body: {"id":"987654"} Documented? DOCUMENTED Expected: 404: no such todo	**Status 404: Not Found** Observed: HTTP 404 — {"errorMessages":["Could not find thing matching value for id"]} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 13	**Test API GET request** /projects/987654/tasks Documented? DOCUMENTED Expected: 404: project does not exist	**Status 200: OK** Observed: HTTP 200 — {"todos":[{"id":"1","title":"scan paperwork","doneStatus":"false","description":"","tasksof":[{"id":"1"}]},{"id":"405","title":"todo created under project","doneStatus":"false","description":"","tasksof":[{"id":"1"}]}]} Result: **Bug** Side note: BUG-04: 200 with todos belonging to other projects	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 14	**Test API DELETE request** /projects/1/tasks/1 Documented? DOCUMENTED Expected: 200: relationship deleted	**Status 200: OK** Observed: HTTP 200 — (empty response body) Result: **As Expected** Side note: Empty response body	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 15	**Test API DELETE request** /projects/1/tasks/1 (repeated) Documented? DOCUMENTED Expected: 404: relationship already deleted (invalid operation)	**Status 404: Not Found** Observed: HTTP 404 — {"errorMessages":["Could not find any instances with projects/1/tasks/1"]} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh

Time	Endpoint Tested	Response	Reference File
Step 16	**Test API GET request** /projects/1/tasks/1 Documented? DOCUMENTED Expected: Swagger documents only DELETE here: 405	**Status 404: Not Found** Observed: HTTP 404 — (empty response body) Result: **Bug** Side note: BUG-08 family: 404 returned instead of 405	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 17	**Test API DELETE request** /projects/987654 Documented? DOCUMENTED Expected: 404 for a missing project	**Status 404: Not Found** Observed: HTTP 404 — {"errorMessages":["Could not find any instances with projects/987654"]} Result: **As Expected**	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Step 18	**Test API GET request** /projects?title=thesis Documented? UNDOCUMENTED Expected: Filtering is not described in the guide; a filtered list is the sensible behaviour	**Status 200: OK** Observed: HTTP 200 — {"projects":[{"id":"202","title":"thesis","completed":"false","active":"false","description":""}]} Result: **As Expected** Side note: Undocumented capability: field filtering	logs/exploratory_session2_projects.log exploratory/session2_projects.sh
Close	**Review findings and cross-reference bugs**	Findings, concerns, risks and new test ideas reviewed and recorded in Section 4; bugs entered in bug_reports.md with the same BUG numbers used there.	bug_reports.md

Section 3: Referenced Files (Scripts, Programs, Screen Shots, Video Clips, Spreadsheets, etc)

Table 3: Descriptions of Referenced Files and Created Scripts

File Name	Description
exploratory/session2_projects.sh	Curl capability script used in this session: demonstrates every capability exercised, in order. Shared helper: exploratory/common.sh.
logs/exploratory_session2_projects.log	Raw session log: the exact status code and response body of every request in Table 2, in step order. This log is the evidence for the Response column (no screenshots were taken; the log is replayable via the script).
session_notes/CH2_Projects_Notes.pdf	This session-notes document. The teacher-template data is contained in this PDF itself: participant/charter information in Section 1, the per-request method, method type, documented status, expected behaviour, expected-vs-observed verdict, observed behaviour, side notes and XML/evidence details in Section 2, and the summary of findings, concerns and test ideas in Section 4.
bug_reports.md	Consolidated bug form and the full bug reports (summary, description, impact, steps to reproduce) for the bugs in Table 4.
tests/test_projects_api.py, tests/test_project_relationships_api.py, tests/test_bugs_expected_behavior.py, tests/test_bugs_actual_behavior.py	Unit tests created from this session's findings, including the expected-behaviour / actual-behaviour pairs for each numbered bug.
Part_A_Report.pdf	Final Part A report: deliverables, findings, suite structure and repository description.

Section 4: Summary of Session Findings

4.1 Summary of Session Findings

Capabilities identified:

- Project CRUD with fields title, completed, active, description (defaults "", false, false).
- Link / unlink existing todos as tasks; create a todo directly under a project (POST /projects/:id/tasks {"title":..}) — the latter undocumented in prose.
- Filter ?title= — undocumented.
- Project <-> todo links are visible from both sides.

What was tried:

Create with/without title and with a bad boolean; amend with POST; replace with PUT and re-read task links; link an existing and a missing todo; unlink twice; GET on a relationship instance; relationships of a missing parent; delete a missing project.

What we learned:

- Core project operations work and project <-> todo links are two-way.
- Title is NOT mandatory for projects, unlike todos and categories (201 with an empty title).
- PUT on a project resets omitted fields AND silently drops all its task relationships (BUG-09).
- GET /projects/987654/tasks returns 200 with unrelated todos instead of 404 (BUG-04).
- GET on a relationship instance returns 404 although swagger documents 405 (BUG-08 family).
- A project may be completed=true and active=true at the same time.

Decisions made:

- Re-read relationships after every PUT, because PUT replace semantics were suspected to touch links.
- Documented "method not allowed" rows were checked with the relationship-instance GET instead of one-off probes.

Table 4: Descriptions of Identified Bugs

Bug	Description
Observation (not filed as a numbered bug) — title is not mandatory for projects	POST /projects with no title returns 201 and stores an empty title, while todos and categories reject a missing title with 400. Validation is inconsistent across entities; whether a title should be mandatory for projects is a business-rule question.
BUG-04 — GET /projects/987654/tasks returns 200 with other data	The parent project does not exist, but the API returns 200 and a list of todos belonging to other projects instead of 404. Clients cannot detect the bad id, and unrelated data is exposed.
BUG-09 — PUT /projects/{id} silently removes task relationships	Replacing a project with PUT resets omitted fields (documented replace semantics) but also drops every task link: GET /projects/1/tasks is empty afterwards although the todos still exist. Silent relationship data loss on an ordinary rename.
BUG-08 (family) — GET on a relationship instance returns 404, not the documented 405	GET /projects/1/tasks/1 returns 404 although swagger documents only DELETE for that URL and 405 for other methods. Clients cannot distinguish “method unsupported” from “not found”.

4.2 Concerns Identified in Session / Areas of Potential Risk

- BUG-04, BUG-08, BUG-09; title optional only for projects (validation is inconsistent across entities).
- Silent data loss of relationships on an ordinary PUT rename.
- Wrong data returned for a missing parent id can mislead or leak to clients.

4.3 New Testing Ideas Identified in Session

- Project completed=true while its tasks are unfinished.
- Deleting a project that has many tasks.
- Linking the same todo to a project twice (duplicate links).
- PUT/POST id-change attempts on projects in XML.